

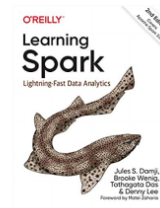
09.1: Apache Spark: Principles



UMD DATA605 - Big Data Systems

09.1: Apache Spark: Principles

- **Instructor:** Dr. GP Saggese, gsaggese@umd.edu
- @References@:
 - Concepts in the slides
 - Academic paper
 - “Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing”, 2012
 - Mastery
 - “Learning Spark: Lightning-Fast Data Analytics” (2nd Edition)
 - Not my favorite, but free

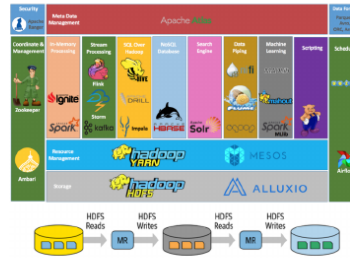


1 / 14

2 / 14: Hadoop MapReduce: Shortcomings

Hadoop MapReduce: Shortcomings

- **Hadoop is hard to administer**
 - Many layers (HDFS, Yarn, Hadoop, ...)
 - Extensive configuration
- **Hadoop is hard to use**
 - Verbose API
 - Limited language support (e.g., Java is native)
 - MapReduce jobs read / write data on disk
- **Large but fragmented ecosystem**
 - No native support for:
 - Machine learning
 - SQL
 - Streaming
 - Interactive computing
 - New systems developed on Hadoop for new workloads
 - E.g., Apache Hive, Storm, Impala, Giraph, Drill



2 / 14

- **Hadoop is hard to administer**
 - Hadoop consists of multiple layers like HDFS, Yarn, and Hadoop itself, making it complex to manage. Each layer requires specific configurations, which can be extensive and challenging to handle.
- **Hadoop is hard to use**
 - The API for Hadoop is verbose, meaning it requires a lot of code to perform tasks, which can be cumbersome for developers. Additionally, Hadoop primarily supports Java, limiting language flexibility. MapReduce jobs in Hadoop read and write data to disk, which can slow down processing due to the I/O operations involved.
- **Large but fragmented ecosystem**
 - Hadoop does not natively support several important functionalities such as machine learning, SQL, streaming, and interactive computing. To address these gaps, new systems like Apache Hive, Storm, Impala, Giraph, and Drill have been developed on top of Hadoop. This fragmentation means users often need to integrate multiple tools to meet their needs, adding complexity to the ecosystem.

3 / 14: (Apache) Spark

(Apache) Spark

- **Open-source**
 - DataBrick monetizes it (\$100B startup in 2025)
- **General processing engine**
 - Large set of operations beyond `Map()` and `Reduce()`
 - Combine operations in any order
 - Computation organized as a DAG, decomposed into parallel tasks
 - Scheduler/optimizer for parallel workers
- **Supports several languages**
 - Java, Scala (preferred), Python supported through bindings
- **Data abstraction**
 - Resilient Distributed Dataset (RDD)
 - DataFrames, Datasets built on RDDs
- **Fault tolerance through RDD lineage**
- **In-memory computation**



3 / 14

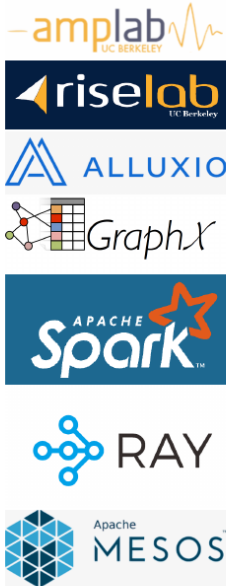
- **Open-source**
 - *Apache Spark* is an open-source platform, meaning its source code is freely available for anyone to use, modify, and distribute. This openness fosters a large community of developers and users who contribute to its development and improvement.
 - *DataBrick monetizes it* by offering commercial services and support around Spark, highlighting its value and potential, as evidenced by its projected valuation as a \$100 billion startup by 2025.
- **General processing engine**
 - Spark is a versatile processing engine that supports a wide range of operations beyond the basic `Map()` and `Reduce()` functions, allowing for more complex data processing tasks.
 - Users can combine operations in any sequence, providing flexibility in data processing workflows.
 - Computations in Spark are organized as a Directed Acyclic Graph (DAG), which breaks down tasks into parallel units, optimizing performance.
 - A scheduler and optimizer manage these parallel tasks, ensuring efficient use of resources.
- **Supports several languages**
 - Spark supports multiple programming languages, including Java, Scala, and Python, making it accessible to a broad range of developers. Scala is often preferred due to its seamless integration with Spark's core.
- **Data abstraction**
 - Spark introduces the concept of Resilient Distributed Datasets (RDDs), which are fault-tolerant collections of elements that can be operated on in parallel.
 - DataFrames and Datasets are higher-level abstractions built on top of RDDs, offering

- more structured data manipulation capabilities.
- **Fault tolerance through RDD lineage**
 - Spark ensures fault tolerance by maintaining the lineage of RDDs, which allows it to reconstruct lost data by reapplying transformations to the original data.
- **In-memory computation**
 - Spark optimizes performance by keeping intermediate data in memory whenever possible, reducing the need for time-consuming disk I/O operations. This capability significantly speeds up data processing tasks.

4 / 14: Berkeley: From Research to Companies

Berkeley: From Research to Companies

- Pathway from lab innovation to startups
 - Students and researchers creating companies from lab systems
 - Focus on data-intensive systems and machine learning
 - Open-source ecosystems enabling broad adoption
- **AMPLab**
 - Collaborative projects creating systems like Spark
 - Industry engagement guiding real-world impact
- **RISELab**
 - Shift to systems supporting AI, security, and automation
 - Platforms like Ray and ML-focused infrastructure



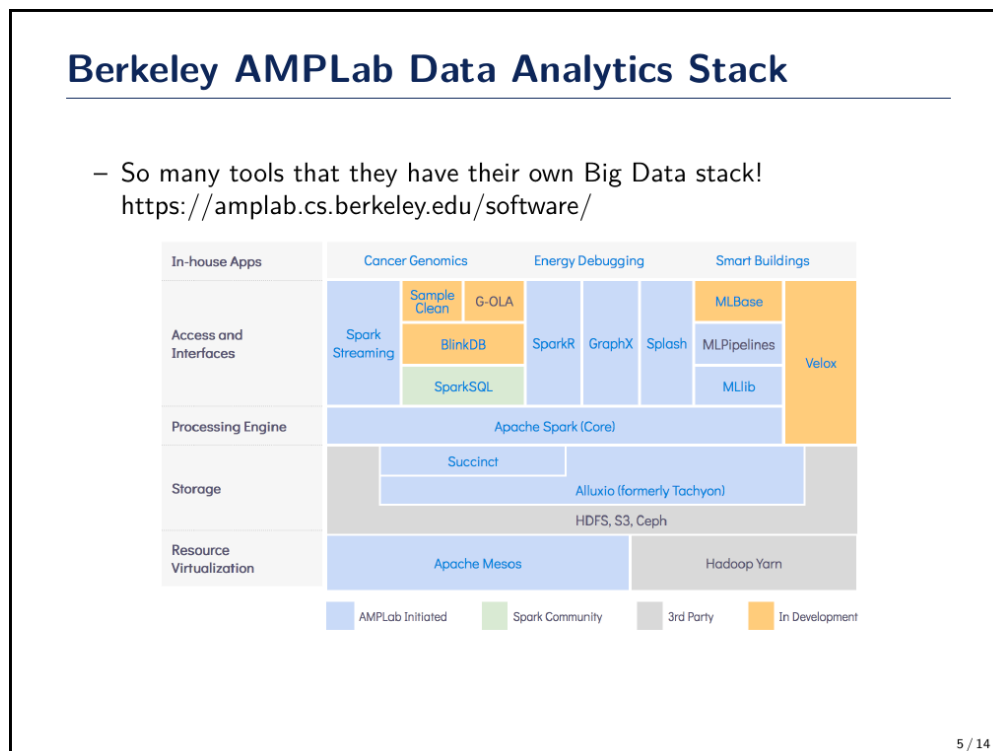
Logos for AMPLab, RISELab, ALLUXIO, GraphX, Apache Spark, RAY, and Apache MESOS.

4 / 14

- **Pathway from lab innovation to startups**
 - *Students and researchers creating companies from lab systems:* This highlights how academic research can lead to the creation of startups. Students and researchers often take their innovative ideas from the lab and turn them into commercial ventures.
 - *Focus on data-intensive systems and machine learning:* The emphasis is on developing systems that handle large amounts of data and utilize machine learning, which are crucial in today's tech landscape.
 - *Open-source ecosystems enabling broad adoption:* By making systems open-source, these innovations can be widely adopted, allowing for community contributions and faster development.
- ****@AMPLab@****
 - *Collaborative projects creating systems like Spark:* AMPLab is known for its collaborative approach, leading to the development of influential systems such as Apache Spark, which is widely used for big data processing.
 - *Industry engagement guiding real-world impact:* By engaging with industry, AMPLab

- ensures that their projects have practical applications and can solve real-world problems.
- ****@RISELab@****
 - *Shift to systems supporting AI, security, and automation*: RISELab focuses on developing systems that support artificial intelligence, enhance security, and enable automation, reflecting current technological trends.
 - *Platforms like Ray and ML-focused infrastructure*: RISELab develops platforms such as Ray, which are designed to support machine learning workloads, providing the necessary infrastructure for AI applications.

5 / 14: Berkeley AMPLab Data Analytics Stack



- **Berkeley AMPLab Data Analytics Stack**
 - The Berkeley AMPLab has developed a comprehensive stack for big data analytics, showcasing a variety of tools and technologies.
 - This stack is designed to handle diverse applications such as *Cancer Genomics*, *Energy Debugging*, and *Smart Buildings*.
- **In-house Apps**
 - These applications are tailored for specific domains, leveraging the stack’s capabilities to address unique challenges in each field.
- **Access and Interfaces**
 - **Spark Streaming**: Enables real-time data processing.
 - **SparkR** and **SparkSQL**: Provide interfaces for data manipulation and querying.
 - **GraphX** and **MLlib**: Facilitate graph processing and machine learning tasks.
 - **Sample Clean**, **G-OLA**, **BlinkDB**, **MLBase**, **MLpipelines**, and **Velox**: Tools in development to enhance data processing and machine learning capabilities.
- **Processing Engine**

- **Apache Spark (Core):** The central processing engine, known for its speed and ease of use in big data processing.
- **Storage**
 - **Succinct** and **Alluxio (formerly Tachyon):** Offer efficient data storage solutions.
 - Compatible with **HDFS**, **S3**, and **Ceph** for flexible storage options.
- **Resource Virtualization**
 - **Apache Mesos** and **Hadoop Yarn:** Manage resources efficiently, allowing for scalable and flexible deployment of applications.
- **Color Coding**
 - The diagram uses color coding to indicate the origin of each component: AMPLab Initiated, Spark Community, 3rd Party, and In Development. This helps in understanding the collaborative nature of the stack's development.

6 / 14: Apache Spark: Introduction

Apache Spark: Introduction

- **Unified stack**
 - Different computation models in a single framework
- **Spark SQL**
 - ANSI SQL compliant
 - Work with structured relational data
- **Spark MLlib**
 - Build ML pipelines
 - Support popular ML algorithms
 - Built on Spark DataFrame
- **Spark Streaming**
 - Handle continually growing tables
 - Treat tables as static
- **GraphX**
 - Manipulate graphs
 - Perform graph-parallel computation

The diagram illustrates Apache Spark as a central 'One computation engine' (represented by a circle with the Spark logo). It connects to various data sources and systems: MySQL, elastic, kafka, DELTA LAKE, redis, mongoDB, cassandra, HBase, Hadoop, and PostgreSQL. Below the central engine, four boxes represent different computation models: 'Spark SQL and DataFrames + Datasets', 'Spark Streaming (Structured Streaming)', 'Machine Learning MLlib', and 'Graph Processing GraphX'. At the bottom, a box labeled 'Spark Core and Spark SQL Engine' contains five sub-boxes for different languages: 'Scala', 'SQL', 'Python', 'Java', and 'R'. The entire diagram is labeled 'General purpose applications' at the bottom.

6 / 14

- **Unified stack**
 - Apache Spark provides a *unified framework* that supports various computation models, making it versatile for different data processing needs.
- **Spark SQL**
 - It is *ANSI SQL compliant*, allowing users to work with structured relational data using familiar SQL queries.
- **Spark MLlib**
 - Facilitates the building of *machine learning pipelines* and supports popular ML algorithms, leveraging Spark DataFrames for efficient data handling.
- **Spark Streaming**
 - Designed to handle *continually growing tables*, treating them as static for processing,

which is useful for real-time data streams.

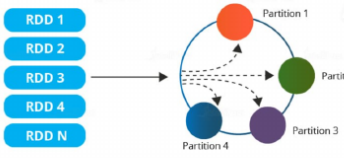
- **GraphX**
 - Enables manipulation and *graph-parallel computation*, allowing for complex graph analytics.
- **Extensibility**
 - Spark can *read from and write to* a wide variety of data sources and backends, enhancing its flexibility and integration capabilities.

The images illustrate Spark's capability as a *single computation engine* that supports *general-purpose applications*. The first image shows Spark's integration with various data sources like MySQL, Kafka, and MongoDB, highlighting its extensibility. The second image outlines the core components and languages supported by Spark, emphasizing its versatility and broad applicability in data processing tasks.

7 / 14: Resilient Distributed Dataset (RDD)

Resilient Distributed Dataset (RDD)

- **Resilient Distributed Dataset (RDD)**
 - Collection of data elements
 - Partitioned across nodes
 - Operated on in parallel
 - Fault-tolerant
 - In-memory / serializable
- **Applications**
 - Best for applying the same operation to all dataset elements (vectorized)
 - Less suitable for asynchronous fine-grained updates to shared state
 - E.g., updating one value in a dataframe
- **Ways to create RDDs**
 - Reference data in external storage
 - E.g., file-system, HDFS, HBase
 - Parallelize an existing collection in your driver program
 - Transform RDDs into other RDDs



7 / 14

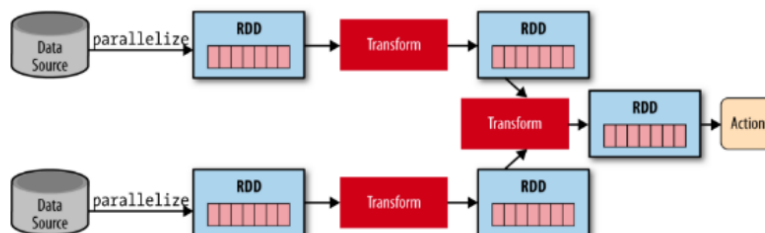
- **Resilient Distributed Dataset (RDD)**
 - *Collection of data elements*: RDDs are essentially groups of data that can be processed.
 - *Partitioned across nodes*: The data is split into parts and distributed across different nodes in a cluster, allowing for efficient processing.
 - *Operated on in parallel*: Operations on RDDs can be performed simultaneously across multiple nodes, speeding up computation.
 - *Fault-tolerant*: RDDs can recover from node failures, ensuring data integrity and reliability.
 - *In-memory / serializable*: Data can be stored in memory for fast access, and it can be serialized for storage or transmission.
- **Applications**

- *Best for applying the same operation to all dataset elements (vectorized)*: Ideal for tasks where the same computation is applied to each data element, like map or filter operations.
- *Less suitable for asynchronous fine-grained updates to shared state*: Not optimal for tasks requiring frequent updates to individual data points, such as modifying a single value in a dataframe.
- **Ways to create RDDs**
 - *Reference data in external storage*: RDDs can be created by accessing data stored in systems like file systems, HDFS, or HBase.
 - *Parallelize an existing collection in your driver program*: You can convert a local collection into an RDD for distributed processing.
 - *Transform RDDs into other RDDs*: RDDs can be transformed into new RDDs using operations like map, filter, and reduce.

8 / 14: Transformations vs Actions

Transformations vs Actions

- **Transformations**
 - Lazy evaluation
 - Compute only when an Action requires it
 - Build a graph of transformations
- **Actions**
 - Aka “materialize”
 - Force calculations on RDDs and return values



8 / 14

- **Transformations**
 - *Lazy evaluation*: Transformations in Spark are not executed immediately. Instead, they are recorded in a lineage graph. This means that Spark will wait until it needs to compute the result, optimizing the process by combining multiple transformations.
 - *Compute only when an Action requires it*: The transformations are only executed when an action is called. This approach helps in optimizing the computation by reducing unnecessary calculations.
 - *Build a graph of transformations*: As transformations are applied, Spark builds a logical plan of operations, which is a directed acyclic graph (DAG). This graph helps in understanding the sequence of operations and optimizing the execution.
- **Actions**

- Aka “materialize”: Actions are operations that trigger the execution of the transformations. They are responsible for producing output from the RDDs.
- Force calculations on RDDs and return values: When an action is called, Spark executes the transformations and returns the result. This is when the actual computation happens, and the results are either returned to the driver program or saved to an external storage system.

The diagram illustrates how data is processed in Spark. Data from a source is parallelized into RDDs, which undergo transformations. These transformations are only executed when an action is applied, resulting in the final output.

9 / 14: Spark Example: Estimate Pi

Spark Example: Estimate Pi

– Goal

- Estimate π using random sampling in the unit square
- Fraction of points inside the unit circle approximates $\pi/4$

– sample generates one random point

- Test membership in the unit circle
- Returns 1 for inside, 0 for outside

– parallelize distributes the sampling task

- Each element in the RDD triggers one call to sample
- “Embarrassingly parallel” computation

– map applies sampling across partitions

- Each worker independently counts hits inside the circle

– reduce aggregates partial sums

- Summing 0 and 1 values yields total count of hits

```
# Estimate pi (compute-intensive task).
# Pick random points in the unit square [(0,0)-(1,1)].
# See how many fall in the unit circle center=(0, 0), radius=1.
# The fraction should be pi / 4.

import random
random.seed(314)

def sample(p):
    x, y = random.random(), random.random()
    in_unit_circle = 1 if x*x + y*y < 1 else 0
    return in_unit_circle

# "parallelize" method creates an RDD.
NUM_SAMPLES = int(1e6)
count = sc.parallelize(range(0, NUM_SAMPLES)) \
    .map(sample) \
    .reduce(lambda a, b: a + b)

approx_pi = 4.0 * count / NUM_SAMPLES
print("pi is roughly %f" % approx_pi)

executed in 386ms, finished 04:27:53 2022-11-23
pi is roughly 3.141400
```

9 / 14

– Goal

- The objective is to estimate the value of π using a method called random sampling within a unit square.
- By calculating the fraction of random points that fall inside a unit circle, we can approximate $\pi/4$. This is because the area of the circle is πr^2 and the area of the square is $4r^2$ when $r = 1$.

– sample Function

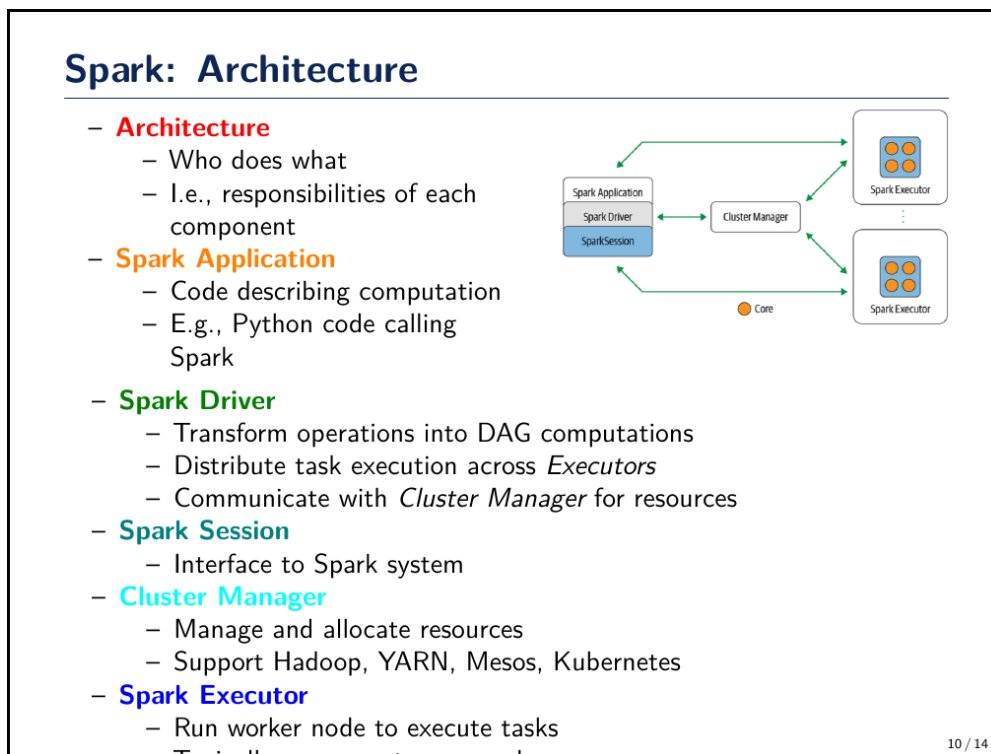
- This function generates a random point within the unit square.
- It checks if the point lies inside the unit circle by evaluating if $x^2 + y^2 < 1$.
- The function returns 1 if the point is inside the circle and 0 otherwise.

– parallelize Method

- This method is used to distribute the task of sampling across multiple nodes in a cluster.
- Each element in the Resilient Distributed Dataset (RDD) corresponds to a call to the sample function.

- The task is “embarrassingly parallel,” meaning it can be easily divided into parallel tasks without dependency.
- **map Function**
 - The **map** function applies the **sample** function to each partition of the RDD.
 - Each worker node independently counts how many points fall inside the circle.
- **reduce Function**
 - The **reduce** function aggregates the results from all partitions.
 - It sums up the 0s and 1s to get the total count of points inside the circle.
- **Code Explanation**
 - The code uses Spark to estimate π by generating a large number of random points.
 - The **parallelize** method creates an RDD with a specified number of samples.
 - The **map** function applies the **sample** function to each element.
 - The **reduce** function sums the results to get the total number of points inside the circle.
 - Finally, it calculates π using the formula $4 \times \text{count} / \text{NUM_SAMPLES}$ and prints the result.

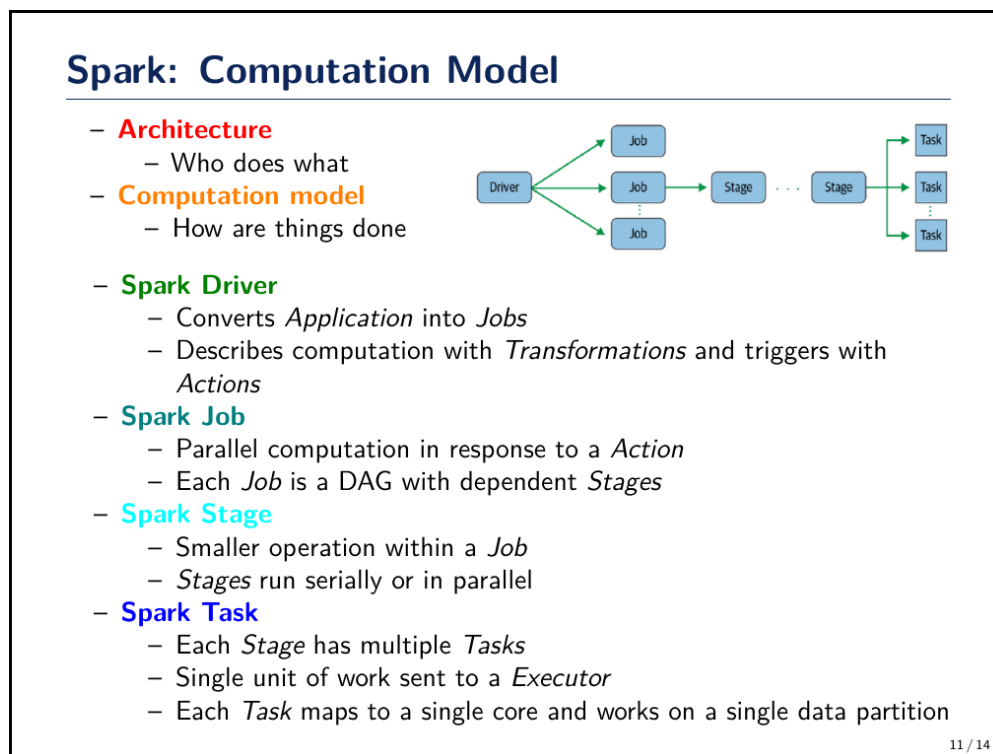
10 / 14: Spark: Architecture



- **Architecture**
 - This section outlines the roles and responsibilities of each component within the Spark architecture. Understanding who does what is crucial for grasping how Spark processes data efficiently.
- **Spark Application**
 - A Spark Application is essentially the code that describes the computation you want to perform. This could be written in languages like Python, where you call Spark functions to process data.

- **Spark Driver**
 - The Spark Driver is responsible for converting operations into Directed Acyclic Graph (DAG) computations. It distributes tasks across Executors and communicates with the Cluster Manager to manage resources.
- **Spark Session**
 - The Spark Session acts as the main entry point to interact with the Spark system. It provides an interface for programming with Spark.
- **Cluster Manager**
 - The Cluster Manager handles resource management and allocation. It supports various cluster management systems like Hadoop, YARN, Mesos, and Kubernetes.
- **Spark Executor**
 - Executors are worker nodes that execute tasks. Typically, there is one executor per node, and they rely on the Java Virtual Machine (JVM) to run tasks.

11 / 14: Spark: Computation Model

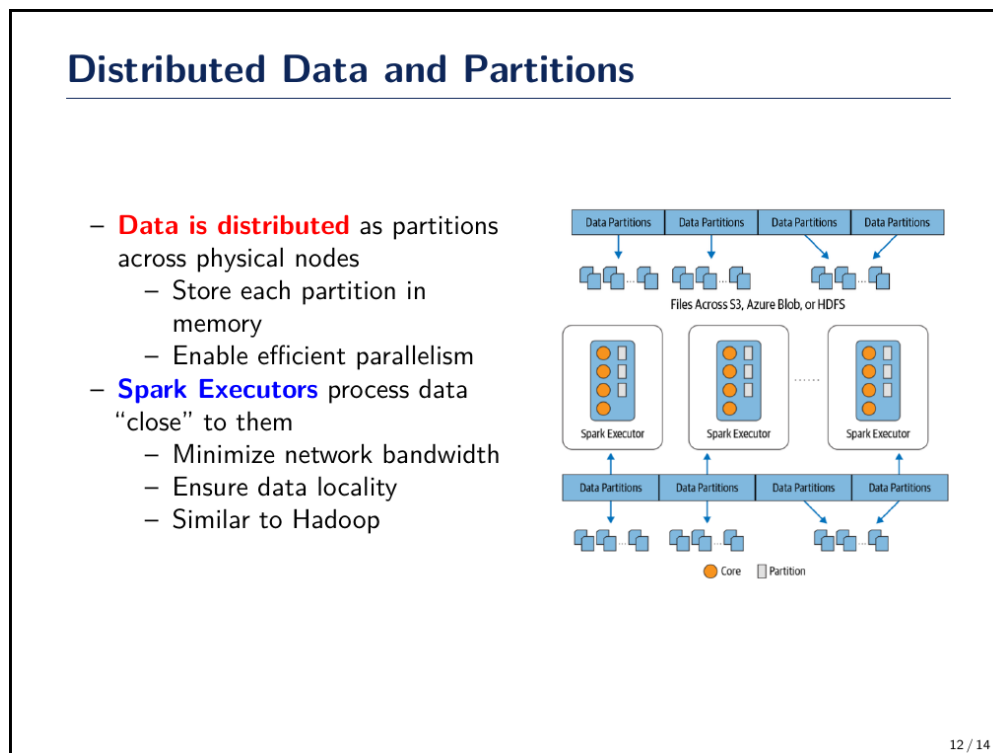


- **Architecture**
 - *Who does what*: In Spark, the architecture defines the roles and responsibilities of different components. The **Driver** is responsible for orchestrating the entire process, while **Executors** perform the actual computation.
- **Computation Model**
 - *How are things done*: Spark uses a distributed computation model that breaks down tasks into smaller units for efficient processing.
- **Spark Driver**
 - Converts an *Application* into multiple *Jobs*. The Driver manages the execution of these jobs by describing the computation through **Transformations** and initiating them with

Actions.

- **Spark Job**
 - Represents a parallel computation triggered by an *Action*. Each job is structured as a Directed Acyclic Graph (DAG) with dependent **Stages** that outline the sequence of operations.
- **Spark Stage**
 - A smaller operation within a job. Stages can run either serially or in parallel, depending on the dependencies and resources available.
- **Spark Task**
 - Each stage is divided into multiple **Tasks**. A task is the smallest unit of work, assigned to an **Executor**. Each task corresponds to a single core and processes a single data partition, ensuring efficient resource utilization.

12 / 14: Distributed Data and Partitions



- **Data is distributed as partitions across physical nodes**
 - Data is broken down into smaller, manageable pieces called *partitions*. These partitions are spread across different physical nodes in a cluster.
 - Each partition is stored in memory, which allows for quick access and processing. This setup is crucial for handling large datasets efficiently.
 - By distributing data this way, systems can achieve *efficient parallelism*, meaning multiple computations can occur simultaneously, speeding up processing time.
- **Spark Executors process data “close” to them**
 - Spark executors are responsible for executing tasks on the data partitions. They are designed to process data that is physically close to them in the network.
 - This proximity minimizes the need for data to travel across the network, reducing *net-*

work bandwidth usage and improving performance.

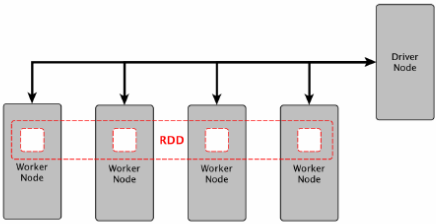
- The concept of *data locality* is similar to what is used in Hadoop, where processing is done near the data to optimize speed and efficiency.

These concepts are visualized in the images, showing how data partitions are distributed across storage systems like S3, Azure Blob, or HDFS, and how Spark executors interact with these partitions.

13 / 14: Parallelized Collections

Parallelized Collections

- Parallelized collections created by calling `SparkContext parallelize()` on an existing collection
- Data spread across nodes
- Number of *partitions* to cut dataset into
 - Spark runs one *Task* per partition
 - Aim for 2-4 partitions per CPU
 - Spark sets partitions automatically based on your cluster
 - Set manually by passing as a second parameter to `parallelize()`



13 / 14

- **Parallelized collections** are created by using the `SparkContext parallelize()` method on an existing collection.
 - This allows you to distribute data across a cluster, enabling parallel processing.
 - The method takes an existing collection (like a list or array) and spreads it across multiple nodes in a cluster.
- **Data spread across nodes** ensures that processing can be done simultaneously on different parts of the data.
 - This is crucial for handling large datasets efficiently.
- **Number of partitions** determines how the dataset is divided.
 - Spark runs one *Task* per partition, meaning each partition is processed independently.
 - The recommended practice is to have 2-4 partitions per CPU to optimize resource usage.
 - Spark can automatically set the number of partitions based on the cluster configuration.
 - Alternatively, you can manually specify the number of partitions by providing a second parameter to the `parallelize()` method.
- The diagram illustrates how data is distributed across worker nodes, with each node handling a portion of the data, known as an RDD (Resilient Distributed Dataset). This setup allows for efficient parallel processing and fault tolerance.

14 / 14: Deployment Modes

Deployment Modes

- Spark can run on several different configurations
 - Components (e.g., Driver, Cluster Manager, and Executors) split on different nodes

@Deployment Mode@	@Where Components Run@	@Notes@
@Local@	Run in a single JVM on one machine	Run Spark on a laptop
@Standalone@	Run in separate JVMs on different machines	Spark's built-in cluster manager
@YARN / Kubernetes@	Run in different pods/containers	Production clusters

14 / 14

- **Deployment Modes:** This section explains the different ways you can set up and run Apache Spark, a powerful tool for big data processing. Each mode has its own setup and use case.
- **Spark can run on several different configurations:** Spark is flexible and can be set up in various ways depending on your needs. The main components of Spark, such as the *Driver*, *Cluster Manager*, and *Executors*, can be distributed across different machines or nodes to optimize performance and resource usage.
- **Local:**
 - **Where Components Run:** All components run in a single Java Virtual Machine (JVM) on one machine.
 - **Notes:** This mode is ideal for development and testing on a personal computer, like a laptop. It's simple and doesn't require a complex setup.
- **Standalone:**
 - **Where Components Run:** Components run in separate JVMs on different machines.
 - **Notes:** This mode uses Spark's built-in cluster manager. It's suitable for small to medium-sized clusters and is relatively easy to set up.
- **YARN / Kubernetes:**
 - **Where Components Run:** Components run in different pods or containers.
 - **Notes:** These are used for production clusters. YARN and Kubernetes are popular resource managers that help manage large-scale deployments, providing scalability and resource efficiency.